

Agent Support Chat System

How real-time chat works: WebSocket + Redis, with no database involved

This document explains, in plain language, how live chat between an Admin and a Field Agent works in the 100 Transcripts platform. It focuses only on the real-time messaging path: the WebSocket connection and Redis storage. No SQL database table is used to store or load chat messages — that is intentional, and this document explains why and how.

1. Simple Definitions

Before the flow, here are the core words used throughout this document, explained simply.

WebSocket — A connection between the browser and the server that stays open. Unlike a normal web request (ask once, get one answer, connection closes), a WebSocket stays connected so either side can send a message at any time, instantly, without refreshing the page.

Django Channels — The library that adds WebSocket support to Django. Django alone only understands normal HTTP requests (GET/POST); Channels lets the same project also accept and manage WebSocket connections.

Consumer — The piece of Python code (AdminAgentConsumer in this project) that runs when someone connects over a WebSocket. It is the WebSocket version of a Django "view" — it decides what happens on connect, on message received, and on disconnect.

Room / Group — A named channel that groups together everyone who should receive the same messages. Only the Admin and the one Agent talking about one specific topic are joined to the same room, so their messages never leak to anyone else.

Redis — A very fast, in-memory data store. It is not a traditional SQL database — it does not use tables, rows, or SQL queries. It stores simple key-to-value data and is built for speed, which is why it is used here for both live delivery and short-term chat history.

Redis key — A unique text label used to store and retrieve one specific piece of data in Redis, similar to a filename. Example: chat:agent:5:app:12 uniquely identifies one conversation's message list.

Cache — A temporary storage layer meant for data that does not need to live forever. Django's cache framework is configured here to use Redis as its backing store.

Token — A signed piece of text issued at login that proves who a user is on later requests, without the server needing to remember a session. If the text has been tampered with or has expired, it fails verification.

2. The Whole System in One Sentence

Chat messages travel over a WebSocket connection (managed by Django Channels), get stored temporarily in Redis for 7 days, and are instantly broadcast to everyone connected to that specific Admin ↔ Agent "room" — no SQL database table is read or written in this path.

3. Why Redis Instead of the SQL Database

The project does have a SQL table that could store chat messages (AgentAdminMessage), but the real-time chat feature intentionally does not use it. Here is the reasoning in simple terms:

- **Speed** — Redis keeps data in memory (RAM), so reading and writing a message is close to instant. A SQL database writes to disk, which is slower and adds delay to something that needs to feel instant, like chat.
- **Simplicity** — a chat conversation is just "a list of messages in order." Redis stores exactly that shape of data directly, with no need to design tables, relationships, or run queries.
- **Built-in expiry** — Redis can automatically delete data after a set time (see Section 6). A SQL table would need a separate scheduled job to clean up old messages.

The trade-off: because nothing is written to the SQL database, chat history is not permanent. This is explained fully in Section 6.

4. Step-by-Step: What Happens When a Message Is Sent

Both the Admin and the Agent use the exact same mechanism — only the `is_from_admin` flag differs. The steps below describe one message being sent by either side.

1

Open the connection

The browser opens a WebSocket connection to the server, e.g. `ws://127.0.0.1:8000/ws/agent/5/app/12/?token=...`. The URL encodes who is chatting (agent 5) and about what (application 12). A signed token is attached so the server can verify the connection is genuine.

2

Server checks the token

`AdminAgentConsumer.connect()` reads the token, verifies its signature, and confirms it belongs to either that agent or an admin. If the token is missing, expired, or does not match, the connection is refused immediately and nothing further happens.

3

Join the room

Once verified, the server builds a room name from the URL, e.g. `agent_5_app_12`, and joins this browser tab's connection to that room. A "room" is just a label Channels uses to know who should receive what.

4

User sends a message

The person types a message and hits Send. The browser sends a small JSON payload over the open connection: `{ "message": "...", "application_id": 12 }`.

5

Server saves it to Redis

The consumer's `receive()` method calls `save_message()`, which appends the new message (with a fresh ID and timestamp) to that conversation's Redis list, trims the list to the most recent 1000 messages, and resets the 7-day expiry timer.

6

Server broadcasts it

The consumer calls `group_send()` on the room. This is Channels' pub-sub mechanism: it instantly pushes the message to every connection currently joined to that room.

7

Everyone in the room receives it live

Both the sender's own tab and the other person's tab (if they are online) receive a `chat_message` event and append it to the on-screen conversation immediately — no page refresh, no re-fetching.

Reading old history works differently — it does not go through the WebSocket at all. When a chat screen first opens, the frontend makes a normal REST call (GET) which calls `get_messages()`, and Redis simply returns the full stored list for that conversation's key.

5. Redis Is Used for Two Different Jobs

It's easy to assume "Redis" means one thing, but this project uses two separate Redis databases (numbered 0 and 1 on the same Redis server) for two very different jobs.

Redis DB	Used By	Job	Keeps Data?
DB 0	CHANNEL_LAYERS (channels_redis)	Real-time delivery only — instantly relays a message from one connection to everyone else in the same room, at the moment it is sent.	No — nothing is kept here after delivery.
DB 1	Django cache (used by redis_chat.py)	Actual chat history — the ordered list of past messages per conversation, so history loads when a chat is opened.	Yes, but only for 7 days (see Section 6).

In short: DB 0 is like a live telephone line (nothing recorded), and DB 1 is like a notepad that gets thrown away after a week of silence.

6. How the 7-Day Expiry Actually Works

Every time a message is saved, the code runs: `cache.set(key, messages, MESSAGE_TTL)`, where `MESSAGE_TTL` is 7 days in seconds. Setting this resets the countdown to a full 7 days from that exact moment, for that one conversation's key only.

```
MESSAGE_TTL = 60 × 60 × 24 × 7    (seconds in 7 days)
```

- An active conversation (messages sent regularly) effectively never expires, because every new message pushes the 7-day clock forward again.
- An inactive conversation with no new messages for 7 straight days has its entire history silently deleted by Redis. There is no warning and no backup.
- Expiry is per conversation, not global. A busy application-specific chat and a quiet general chat with the same agent expire completely independently, because each has its own separate Redis key.
- Each conversation's message list is also capped at the most recent 1000 messages (`messages[-MAX_MESSAGES:]`). Once a conversation passes 1000 messages, the oldest ones are dropped even inside the 7-day window.

7. How Different Conversations Stay Separate

Isolation between agents and between chat types is achieved purely through naming — both for the live WebSocket room and for the Redis storage key. There is no shared "inbox" table to accidentally query across users.

	Application Chat	General Chat
URL parameter	app_id is present	app_id is absent
WebSocket room name	agent_<id>_app_<appld>	agent_<id>_general
Redis key	chat:agent:<id>:app:<appld>	chat:agent:<id>:general

Agent A can never receive Agent B's messages, because their browser connections are joined to entirely different Channels groups, and their messages live under entirely different Redis keys. The same agent's general chat and application-specific chat also never mix, for the same reason.

8. Authentication on the WebSocket Connection

A browser cannot attach a custom "Authorization" header when opening a WebSocket the way it can for a normal fetch() request. To work around this, the signed token is passed as a query parameter directly in the connection URL:

```
ws://127.0.0.1:8000/ws/agent/5/app/12/?token=<signed-token-here>
```

When the connection request arrives, connect() reads this token from the URL's query string and checks it in one of two ways:

- If it is a valid Agent token, the Agent ID inside the token must match the agent_id in the URL — an agent cannot use their own token to open a different agent's chat room.
- If it is not a valid Agent token, it is checked as an Admin token instead.
- If neither check succeeds, the connection is closed immediately with code 4401, before it ever joins a room or is accepted. No message can be sent or received on a rejected connection.

As an additional safeguard, who a message is "from" (Admin or Agent) is decided by which kind of verified token opened that connection — not by a flag the browser sends in the message itself. This prevents one side from impersonating the other.

9. Visual Summary of the Flow

Admin Browser	AdminAgentConsumer	Redis (DB 1 — cache)	Agent Browser
1. Sends over WebSocket: {message, application_id}	→ receives, verifies sender is trusted		
	2. calls save_message()	→ appends message, resets 7-day timer	
	3. calls group_send() to the room		
4a. Own message appears instantly			4b. Message appears instantly (if online)
5. Later: opening the chat again	(REST endpoint, not WebSocket)	→ get_messages() returns the full stored list	

Both Admin-to-Agent and Agent-to-Admin messages follow this exact same path — the consumer code does not treat either side specially, it just records which verified connection sent the message.

10. What Lives in Redis vs. the SQL Database

To be precise about scope: only the live chat text uses Redis. Everything else in the Agent workflow still uses the normal SQL database, exactly as before.

Data	Where it lives	Permanent?
Agent profile, credentials, workload	SQL database (Agent table)	Yes
Assignment status / workflow progress	SQL database (AgentAssignment table)	Yes
Status-change activity log	SQL database (TrackingHistory table)	Yes
Live chat message text	Redis DB 1 only	No — 7-day rolling window

Prepared as a plain-language reference for the Agent Support Chat system's real-time messaging path — WebSocket connection handling (*consumers.py*), Redis-backed history (*redis_chat.py*), and the signed-token authentication added to the WebSocket connection.